

Project Design Phase-II

Technology Stack (Architecture & Stack)

Date	19 July 2026
Team ID	
Project Name	AI-Powered Debt Relief & Financial Recovery Platform
Maximum Marks	4 Marks

Technical Architecture:

The architecture below shows how a borrower's loan data flows from the React frontend, through the FastAPI backend and its rule-based financial-analysis engine, to Google Gemini for AI-generated negotiation letters, and back to the borrower — with SQLite persisting loans and negotiation history throughout.

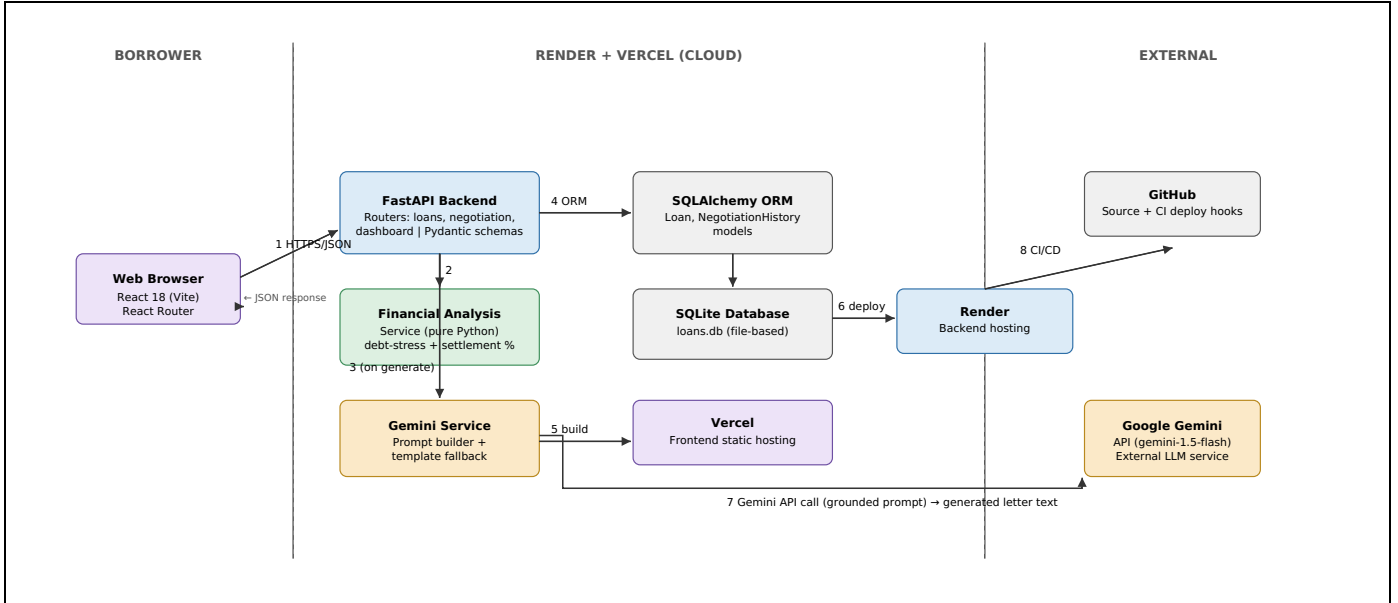


Table-1: Components & Technologies

S.No	Component	Description	Technology
1.	User Interface	Borrower-facing web app: add loan, view analysis, generate letters, dashboard	React 18 (Vite), React Router v6, custom CSS
2.	Application Logic-1	REST API, request/response validation, routing	FastAPI, Pydantic, Uvicorn
3.	Application Logic-2	Rule-based debt-stress & settlement-percentage engine	Python (financial_analysis.py)
4.	Application Logic-3	Negotiation letter prompt construction + fallback templating	Python (gemini_service.py)
5.	Database	Stores Loan and NegotiationHistory records	SQLite via SQLAlchemy ORM
6.	Cloud Database	Same SQLite file, persisted on the backend host	SQLite file on Render instance disk
7.	File Storage	No binary/file storage required by the current scope	Not applicable
8.	External API-1	Generates personalized negotiation letters from loan & borrower data	Google Gemini API (gemini-1.5-flash)
9.	External API-2	None currently integrated	Not applicable
10.	Machine Learning Model	Debt-stress scoring and settlement recommendation are deterministic/rule-based, not ML; letter generation uses Gemini's hosted LLM rather than a locally trained model	Google Gemini (hosted LLM)
11.	Infrastructure (Server / Cloud)	Local Server Configuration: Uvicorn dev server on localhost during development. Cloud Server Configuration: FastAPI on Render (backend), static React build on Vercel (frontend).	Render, Vercel

Table-2: Application Characteristics

S.No	Characteristics	Description	Technology
1.	Open-Source Frameworks	Core web framework, ORM, and UI library are all open source	FastAPI, SQLAlchemy, React, Vite
2.	Security Implementations	CORS restricted to the deployed frontend origin; Gemini API key stored as an environment variable and never committed (.env gitignored); all inputs validated via Pydantic schemas before processing	FastAPI CORS middleware, Pydantic validation, environment-based secrets
3.	Scalable Architecture	Three-tier separation (React UI → FastAPI service layer → SQLite/ORM), with the financial-analysis and Gemini services isolated as independent modules so either can be swapped or scaled separately	FastAPI routers/services pattern
4.	Availability	Frontend and backend are deployed as separate managed services (Vercel, Render) so a redeploy of one does not take down the other	Render, Vercel managed hosting
5.	Performance	A template-based fallback generates negotiation letters instantly if the Gemini API is unavailable or unconfigured, keeping the negotiation flow responsive; SQLite keeps read/write latency low for the current data scale	Template fallback in gemini_service.py, SQLite

References:

<https://c4model.com/>
<https://www.ibm.com/cloud/architecture>
<https://aws.amazon.com/architecture>